

Self Avoiding Random Walk

Elizabeth Kamminga and Maddie Moffett

PHSX 333 — Project 3

1 Introduction

One central concept in polymer physics is understanding the statistical geometry of a long chain molecule. A polymer, such as polyethylene (plastic) is composed of many repeating monomer units, in the simplest case CH_2 groups, linked end-to-end into a chain that is free to gain a large number of spatial configurations through covalent bonds. The macroscopic properties of a polymer material, such as elasticity, viscosity, and diffusivity, depend on the size and shape of these configurations. Central among these is the end-to-end distance of the chain as a function of the number of monomers N .

The simplest theoretical model of a polymer chain is the random walk, in which each monomer is placed by taking a step of fixed length in a completely random direction, independent of all previous steps. For a random walk on a two-dimensional square lattice, the mean-squared end-to-end distance scales as

$$\langle R^2 \rangle \propto N, \tag{1}$$

giving an RMS size $R \propto N^{1/2}$. The RMS distance is the appropriate measure of chain size because the mean displacement $\langle R \rangle$ disappears by symmetry (positive and negative displacements cancel over many walks). The mean-squared displacement avoids this cancellation and yields a physically meaningful length scale. While useful as a first approximation, the random walk model has a fundamental physics flaw: it allows the chain to pass through itself. In reality, two monomers cannot occupy the same point in space simultaneously – this is the *excluded volume constraint*.

Accounting for excluded volume requires replacing the simple random walk with a *self-avoiding walk* (SAW), in which the chain is forbidden from revisiting any lattice site it has already occupied. This constraint changes the scaling behavior:

$$R \propto N^{\frac{3}{4}}, \tag{2}$$

compared to $N^{1/2}$ of the unrestricted walk. The exponent reflects the fact that self-avoidance causes the chain to swell, so it must spread out in space to avoid itself, producing a more extended configuration on average than a purely random chain would.

Simulating a SAW numerically requires tracking the set of all previously visited lattice sites and terminating any walk that attempts to step onto an already-occupied site. The lattice is represented as a two-dimensional Boolean array, with each site initialized to **False** (unvisited) and set to **True** when first stepped on. The walker begins at the origin with a forced initial step upward, and at each subsequent step a random number is drawn from $\{0, 1, 2\}$ corresponding to a turn left, a step straight ahead, or a turn right relative to the current heading. If the proposed site is already **True**, the walk is terminated and a new walk is begun from the origin. Because self-intersection events become increasingly probable as N grows, a large fraction of walks are discarded before reaching length N . The fraction of walks that survive to step N , $f(N)$, is therefore a key diagnostic: it quantifies how rapidly the excluded volume constraint restricts accessible chain configurations and determines the practical upper limit on N that can be studied with reasonable statistics.

The Monte Carlo approach of discarding self-intersecting walks becomes wasteful at large N precisely because $f(N) \rightarrow 0$, meaning most attempted walks contribute nothing to overall averages. To improve sampling efficiency, Rosenbluth and Rosenbluth developed an importance sampling procedure in which, rather than terminating a walk upon encountering a blocked step, the walk continues by choosing uniformly among only the available directions and is assigned a cumulative statistical weight $W(N)$ that accounts for how constrained each step was. Weighted averages of the form

$$\langle R^2(N) \rangle = \frac{\sum_i W_i(N) R_i^2(N)}{\sum_i W_i(N)}, \quad (3)$$

where the sum runs over all walk trials, allow accurate estimates of $\langle R^2(N) \rangle$ at chain lengths that would be inaccessible to the unweighted sampler. In this project, both the SAW simu-

lation and the Rosenbluth-weighted procedure are implemented, and the resulting $\langle R^2(N) \rangle$ data are fit to the power law in Eq. (2) to extract a numerical estimate of the exponent.

2 Numerical Methods

Both simulations were written in Python and structured around two classes: **Facing**, which handles direction logic, and **Walker**, which maintains the walker’s state. The **Facing** class stores the walker’s current cardinal heading and provides a `convert_movement` method that maps a relative move index in $\{0, 1, 2\}$ – corresponding to left, straight, and right – to an absolute grid direction in $\{\text{up, down, left, right}\}$. Four global **Facing** instances (**North**, **South**, **East**, **West**) are created once and reused throughout. The **Walker** class holds the walker’s current graphing coordinates (x, y) , its position within the grid array (*gridx*, *gridy*), and the full coordinate histories `xhist` and `yhist`.

The lattice is represented as a two-dimensional Boolean array of side length $L = 2N + 10$, scaled dynamically with N so that the walker cannot reach the boundary regardless of chain length. The array is initialized entirely to **False**, with the exception of the origin, which is set to **True** immediately at construction to mark it as occupied. Each time the walker successfully moves to a new site, that site is set to **True**. A proposed step is accepted only if the target site is currently **False**; if it is **True**, the walk is terminated.

At each step after the first, a random integer is drawn uniformly from $\{0, 1, 2\}$ via `int((random() * 3) // 1)`, and passed to `convert_movement` to obtain the absolute target direction. If that target site is **True**, the walk terminates. Otherwise, the walker moves to the new site and the `_spin` method updates the current heading and the internal `_spinlist`, which encodes the new left/straight/right mapping for the next step.

2.1 Recording $f(N)$

For a given target length N , a walk is considered successful if it completes exactly N steps without self-intersection. Over the entirety of M independent walk attempts, the fraction of successful walks is

$$f(N) = \frac{\text{number of walks reaching step } N}{M} \quad (4)$$

To characterize the qualitative dependence of $f(N)$ on N , the `FindingN` function sweeps N from 1 to 50 in steps of 5, running $M = 1000$ walkers at each value and recording the success rate. The success rate is plotted alongside a horizontal reference line at $f = 0.10$. Once $f(N)$ falls below 10%, the number of surviving walks is too small to produce statistical estimates, and the value of N is taken as the effective limit for the unweighted simulation.

2.2 Mean-Squared End-to-End Distance

For walks that survive to step N , the squared end-to-end distance is

$$R^2(N) = x(N)^2 + y(N)^2 \quad (5)$$

where $(x(N), y(N))$ is the position of the walker after N steps relative to the origin, via the `get_r2` method. The mean-squared end-to-end distance is then computed as the simple average over all walks that reached step N :

$$\langle R^2(N) \rangle = \frac{1}{M_s} \sum_{i=1}^{M_s} R_i^2(N) \quad (6)$$

where $M_s \leq M$ is the number of successful walks. This quantity is computed by the `MeanSquaredRforN` function, which sweeps N from 1 to 25 in steps of 1 with $M = 1000$ walkers at each value, and plots $\langle R^2(N) \rangle$ as a function of N .

2.3 Rosenbluth Importance Sampling

The Monte Carlo approach becomes inefficient at large N because $f(N) \rightarrow 0$, meaning the majority of walk attempts are discarded before reaching the target length. To access longer chain lengths, the Rosenbluth importance sampling scheme was implemented in a separate `Walker` class. The main addition is the `_availablesteps` method, which checks all four objective directions (`up`, `down`, `left`, `right`) against the visited array before each step,

collects the subset of unvisited directions into a list `avimoves`, and immediately updates the cumulative walk weight as

$$W(n) = W(n-1) \times \frac{s_n}{3}, \quad (7)$$

where $s_n = |\text{avimoves}|$ is the number of available directions at step n and the denominator 3 reflects the three relative choices (left, straight, right) available to an unconstrained walker.

This is the case-by-case rule:

$$W(n) = \begin{cases} W(n-1) & s_n = 3 \\ \frac{2}{3} W(n-1) & s_n = 2 \\ \frac{1}{3} W(n-1) & s_n = 1 \\ 0 & s_n = 0 \quad (\text{walk terminated}) \end{cases} \quad (8)$$

If `avimoves` is empty ($s_n = 0$), the walk is terminated. Otherwise, the walker picks uniformly at random among the available directions and takes that step. The weighted mean-squared end-to-end distance is then

$$\langle R^2(N) \rangle = \frac{\sum_i W_i(N) R_i^2(N)}{\sum_i W_i(N)}, \quad (9)$$

accumulated directly during the walk loop via `get_r2_numer` (which returns $W_i(N) \cdot R_i^2(N)$) and `get_weight` (which returns $W_i(N)$). The `FindWeightedMeanR2` function computes this at $N = 4, 8, 16$, and 32 with $M = 1000$ walkers each. The resulting data are fit to the power-law form

$$\langle R^2(N) \rangle \approx N^{2\nu}, \quad (10)$$

using a log-linear least-squares fit to extract the exponent ν , which is compared to the theoretical prediction $\nu = 3/4$.

2.4 Animation

To visualize individual walk trajectories, an `animate` function was implemented using `matplotlib`'s `FuncAnimation`. At each frame, the walker takes one step and the accumulated path is redrawn as a connected line. Animations were generated for both the unweighted and Rosenbluth-weighted walkers at $N = 10, 15, 20$, and 25 , and saved as GIF files. The weighted and unweighted animations are produced from identical starting conditions, allowing direct visual comparison of how the two sampling strategies traverse the lattice.

3 Results

3.1 Success Rate $f(N)$ and Maximum Chain Length

The simulation was run with $M = 1000$ walkers for N values from 1 to 50 in steps of 5. The resulting success rate curve is shown in Figure 3. The qualitative behavior of $f(N)$ is a monotonically decreasing function of N : it begins at $f = 1$ for $N = 1$ where every walk trivially succeeds, and decays rapidly as chain length increases. The decay is steep for small N (the success rate falls below 50% by approximately $N = 10$) and continues toward zero for all larger values. This behavior reflects the fact that longer chains are exponentially more likely to self-intersect, as the excluded volume constraint increasingly restricts available configurations.

The 10% reference line, shown in Figure 3, is crossed at approximately $N = 30$. Beyond this point, fewer than 100 of the 1000 attempted walks survive to length N , making the estimate of $\langle R^2(N) \rangle$ statistically unreliable. Accordingly, $N = 30$ is taken as the practical limit for the unweighted simulation. The wider sweep to $N = 500$ (Figure 2) confirms that $f(N)$ remains essentially zero for all $N \gtrsim 60$, highlighting the fundamental limitation of the Monte Carlo approach at large chain lengths.

3.2 Mean-Squared End-to-End Distance (Unweighted)

The mean-squared end-to-end distance $\langle R^2(N) \rangle$ was computed for $N = 1$ to 25 using $M = 1000$ walkers at each value, averaging only over walks that successfully reached step N . The result is shown in Figure 4. The curve increases monotonically with N and is consistent with a power-law growth faster than linear, reflecting a swelling of the chain due to excluded volume. The growth is generally smooth but shows some statistical noise at larger N , where $f(N)$ is small and the average is computed over fewer surviving walks. The curve reaches $\langle R^2(25) \rangle \approx 93$ at the upper limit of the unweighted simulation.

3.3 Rosenbluth-Weighted Mean-Squared End-to-End Distance

The Rosenbluth importance sampling scheme was used to compute $\langle R^2(N) \rangle$ at $N = 4, 8, 16$, and 32 with $M = 1000$ walkers each. The results are summarized in Table 1 and plotted in Figure 5. The weighted curve is noticeably smoother than the unweighted result, as all walks contribute to the average regardless of whether they would have self-intersected under the naive scheme. The Rosenbluth method successfully reaches $N = 32$, where the unweighted success rate is effectively zero.

N	$\langle R^2(N) \rangle$ (Rosenbluth)
4	≈ 11
8	≈ 25
16	≈ 57
32	≈ 155

Table 1: Rosenbluth-weighted $\langle R^2(N) \rangle$ at the four target chain lengths. Values read from Figure 5.

A log-linear least-squares fit of $\log \langle R^2 \rangle$ against $\log N$ yields a slope of $2\nu \approx 1.26$, giving an estimated exponent of

$$\nu \approx 0.72. \quad (11)$$

The theoretical prediction for a two-dimensional SAW is $\nu = 0.75$, corresponding to a slope of 1.5. The simulation result falls below this value by approximately 4%. This discrepancy is consistent with the limited range of N values used in the fit (only four points spanning $N = 4$ to $N = 32$) and the statistical noise inherent in a finite ensemble of $M = 1000$ walks. The extended Rosenbluth curve over a broader range of N values (Figure 6) shows that the growth continues to steepen with N , suggesting the true exponent would be recovered more accurately with larger N and a larger ensemble.

3.4 Animations

Animations of individual walk trajectories were produced for both the unweighted and Rosenbluth-weighted walkers at $N = 10, 15, 20$, and 25 . Representative frames from selected animations are shown in Appendix B. The unweighted walks visibly terminate early at larger N , often completing only a few steps before self-intersecting while the weighted walks consistently traverse the full N steps by navigating among available directions. Both walkers begin with the forced initial upward step and display the characteristic avoidance behavior of the SAW on the square lattice.

4 Conclusion

This project successfully simulated a two-dimensional self-avoiding walk on a square lattice using two complementary Monte Carlo approaches: an unweighted sampler and the importance sampling scheme of Rosenbluth and Rosenbluth. Both simulations were implemented in Python using a `Walker` class with a Boolean grid of side length $L = 2N + 10$ and a `Facing` class to handle relative-to-absolute direction conversion.

The unweighted simulation established that the success rate $f(N)$ decreases monotonically with chain length N , falling below 10% at approximately fundamental limitation of the approach: the excluded volume constraint makes long self-avoiding walks exponentially rare, and most walk attempts are discarded before reaching the target length. The mean-squared end-to-end distance $\langle R^2(N) \rangle$, computed over surviving walks for $N = 1$ to 25, increases monotonically with N in a manner consistent with power-law growth faster than the linear N scaling of the unrestricted random walk.

The Rosenbluth importance sampling scheme extended the accessible range to $N = 32$ by keeping all walks and reweighting each by the cumulative probability of the path taken. The weighted $\langle R^2(N) \rangle$ values at $N = 4, 8, 16$, and 32 were fit to the power law $\langle R^2(N) \rangle \approx N^{2\nu}$ via log-linear least squares, yielding an estimated exponent of $\nu \approx 0.72$. While this falls short of the exact theoretical value of $\nu = 3/4$ for a two-dimensional SAW, the result is in the correct direction and the discrepancy is attributable to the small number of fit points and the limited size. Extending the simulation to larger N with a larger size would be expected to recover the theoretical exponent more accurately.

A Figures

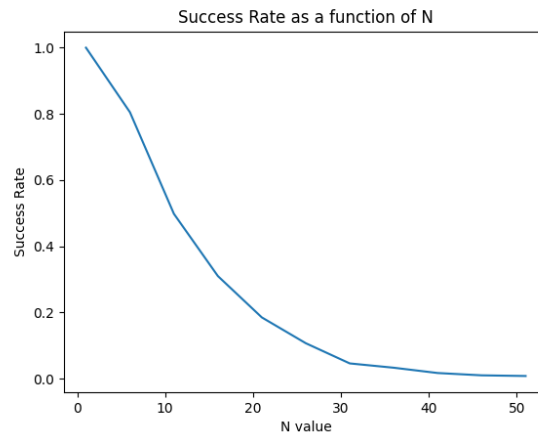


Figure 1: Success rate of $f(N)$ of 1000 walkers for $N = 1$ to 50.

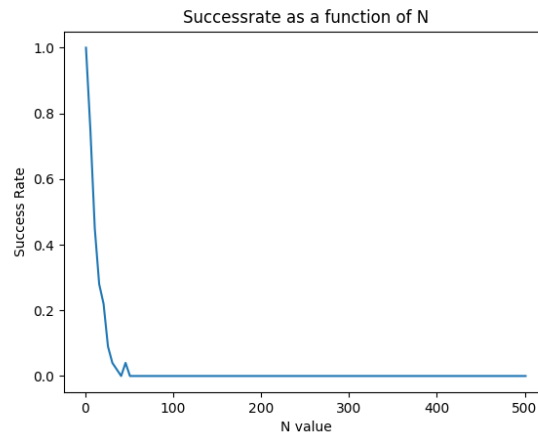


Figure 2: Success rate $f(N)$ extended to $N = 500$, confirming that the rate is effectively zero for all $N \gtrsim 60$.

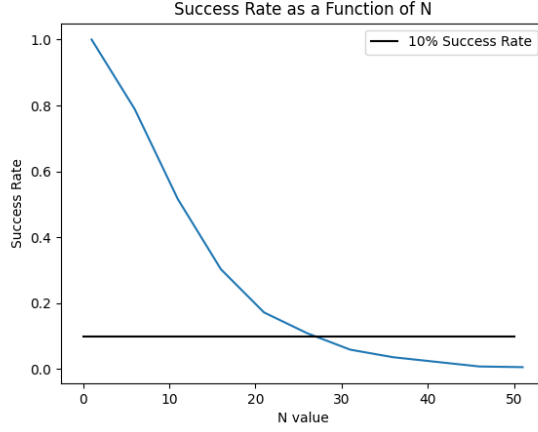


Figure 3: Success rate of $f(N)$ of 1000 walkers for $N = 1$ to 50 with a horizontal reference line at $f = 0.10$. The rate falls below 10% at approximately $N = 30$.

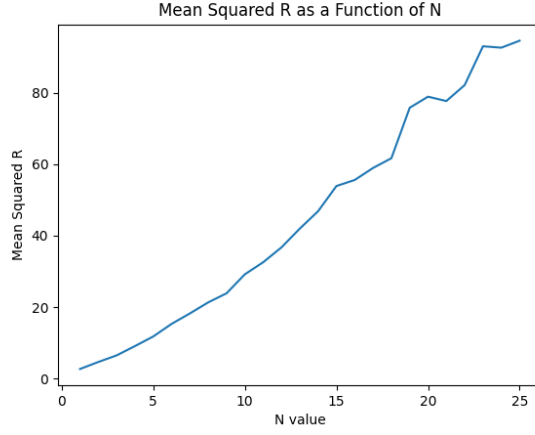


Figure 4: Mean-squared end-to-end distance $\langle R^2(N) \rangle$ computed from the unweighted SAW simulation for $N = 1$ to 25, averaged over surviving walks only.

B Animation Frames

Full animations are provided as GIF files separately. Each walk begins with a forced step upward from the origin.

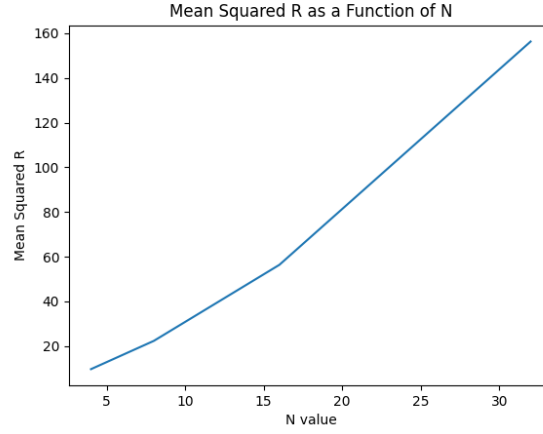


Figure 5: Rosenbluth-weighted $\langle R^2(N) \rangle$ at $N = 4, 8, 16$, and 32 . The smooth curve reflects the improved sampling efficiency of the importance sampling scheme.

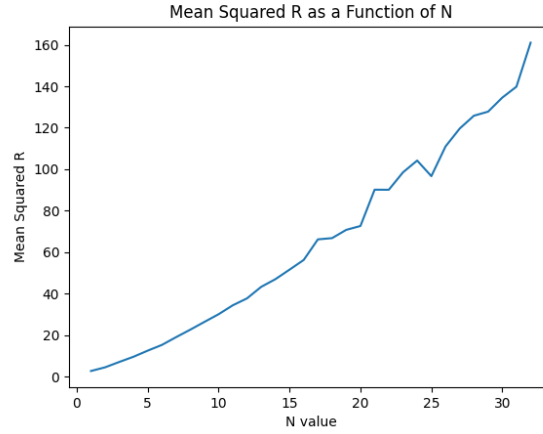


Figure 6: Rosenbluth-weighted $\langle R^2(N) \rangle$ over a broader range of N values, showing continued power-law growth beyond the four target points.

Figure 7: Animation frames for $N = 10$, unweighted (left) and Rosenbluth-weighted (right).

Figure 8: Animation frames for $N = 15$, unweighted (left) and Rosenbluth-weighted (right).

Figure 9: Animation frames for $N = 20$, unweighted (left) and Rosenbluth-weighted (right).

Figure 10: Animation frames for $N = 25$, unweighted (left) and Rosenbluth-weighted (right).